JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class InputStreamReader

java.lang.Object
    java.io.Reader
        java.io.InputStreamReader

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `Readable`

Direct Known Subclasses:

`FileReader`

```
public class InputStreamReader
extends Reader
```

An InputStreamReader is a bridge from byte streams to character streams: It reads bytes and decodes them into characters using a specified `charset`. The charset that it uses may be specified by name or may be given explicitly, or the platform's default charset may be accepted.

Each invocation of one of an InputStreamReader's read() methods may cause one or more bytes to be read from the underlying byte-input stream. To enable the efficient conversion of bytes to characters, more bytes may be read ahead from the underlying stream than are necessary to satisfy the current read operation.

For top efficiency, consider wrapping an InputStreamReader within a BufferedReader. For example:

```
BufferedReader in
  = new BufferedReader(new InputStreamReader(System.in));
```

Since:

JDK1.1

See Also:

BufferedReader, InputStream, Charset

Field Summary

Fields inherited from class java.io.Reader
lock

Constructor Summary

Constructors

Constructor	Description
InputStreamReader(InputStream in)	Creates an InputStreamReader that uses the default charset.
InputStreamReader(InputStream in, Charset cs)	Creates an InputStreamReader that uses the given charset.
InputStreamReader(InputStream in, CharsetDecoder dec)	Creates an InputStreamReader that uses the given charset decoder.
InputStreamReader(InputStream in, String charsetName)	Creates an InputStreamReader that uses the named charset.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
void	close()	Closes the stream and releases any system resources associated with it.
String	getEncoding()	

		Returns the name of the character encoding being used by this stream.
int	**read**()	Reads a single character.
int	**read**(char[] cbuf, int offset, int length)	Reads characters into a portion of an array.
boolean	**ready**()	Tells whether this stream is ready to be read.

Methods inherited from class java.io.Reader

mark, markSupported, read, read, reset, skip

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

InputStreamReader

```
public InputStreamReader(InputStream in)
```

Creates an InputStreamReader that uses the default charset.

Parameters:

in - An InputStream

InputStreamReader

```
public InputStreamReader(InputStream in,
                         String charsetName)
                  throws UnsupportedEncodingException
```

Creates an InputStreamReader that uses the named charset.

Parameters:

in - An InputStream

charsetName - The name of a supported charset

Throws:

UnsupportedEncodingException - If the named charset is not supported

InputStreamReader

```
public InputStreamReader(InputStream in,
                         Charset cs)
```

Creates an InputStreamReader that uses the given charset.

Parameters:

in - An InputStream

cs - A charset

Since:

1.4

InputStreamReader

```
public InputStreamReader(InputStream in,
                         CharsetDecoder dec)
```

Creates an InputStreamReader that uses the given charset decoder.

Parameters:

in - An InputStream

dec - A charset decoder

Since:

1.4

Method Detail

getEncoding

```
public String getEncoding()
```

Returns the name of the character encoding being used by this stream.

If the encoding has an historical name then that name is returned; otherwise the encoding's canonical name is returned.

If this instance was created with the InputStreamReader(InputStream, String) constructor then the returned name, being unique for the encoding,

may differ from the name passed to the constructor. This method will return `null` if the stream has been closed.

Returns:

`The historical name of this encoding, or null if the stream has been closed`

See Also:

`Charset`

read

```
public int read()
         throws IOException
```

Reads a single character.

Overrides:

`read in class Reader`

Returns:

`The character read, or -1 if the end of the stream has been reached`

Throws:

`IOException - If an I/O error occurs`

read

```
public int read(char[] cbuf,
                int offset,
                int length)
         throws IOException
```

Reads characters into a portion of an array.

Specified by:

`read in class Reader`

Parameters:

`cbuf - Destination buffer`

`offset - Offset at which to start storing characters`

`length - Maximum number of characters to read`

Returns:

`The number of characters read, or -1 if the end of the stream has been reached`

Throws:

IOException - If an I/O error occurs

ready

```
public boolean ready()
              throws IOException
```

Tells whether this stream is ready to be read. An InputStreamReader is ready if its input buffer is not empty, or if bytes are available to be read from the underlying byte stream.

Overrides:

ready in class Reader

Returns:

True if the next read() is guaranteed not to block for input, false otherwise. Note that returning false does not guarantee that the next read will block.

Throws:

IOException - If an I/O error occurs

close

```
public void close()
           throws IOException
```

Description copied from class: Reader

Closes the stream and releases any system resources associated with it. Once the stream has been closed, further read(), ready(), mark(), reset(), or skip() invocations will throw an IOException. Closing a previously closed stream has no effect.

Specified by:

close in interface Closeable

Specified by:

close in interface AutoCloseable

Specified by:

close in class Reader

Throws:

C

```
IOException - If an I/O error occurs
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.